



University of Piraeus

School of Information and Communication Technologies

Department of Digital Systems

Level: Master's Programme of Studies

Wireless and Mobile Network Security

Android-Use Case 2

Supervising Professors: Christos Xenakis and Georgios Efthymoglou
(Grigorios Papoutsis)

Ioannis Kalaitzidis

ioannis_kalaitzidis@ssl-unipi.gr

Piraeus

Abstract

This assignment investigates authentication mechanism vulnerabilities in Android applications, analysing three different methodological approaches: static analysis and Smali code patching (Apktool), reverse engineering of native code (Ghidra), and runtime manipulation via dynamic instrumentation (Frida).

Abstract

This study investigates authentication mechanism vulnerabilities in Android applications by exploring three distinct bypass methodologies. Through static analysis using JADX and APKTool (Smali Patching) and native library reverse engineering with Ghidra, it is demonstrated that mere credential obfuscation is insufficient security. Furthermore, utilizing the Frida framework for dynamic instrumentation, runtime memory manipulation is performed to bypass all client-side authentication controls. The results confirm that client-side security measures are inherently fragile, highlighting the critical necessity for robust server-side validation and advanced cryptographic implementations in mobile application development.

Contents

Introduction.....	
Practical part of the assignment.....	
Bibliography.....	24

Introduction

The rapid evolution of mobile operating systems, with Android dominant, has transformed smartphones into powerful computing devices storing sensitive personal data.

This assignment focuses on the analysis and practical evaluation of the security mechanisms of an Android application (project02.apk), using three complementary techniques.

During the analysis, three distinct methodological pillars are examined:

1. **Static analysis: Through the reconstruction and modification of Smali code, the vulnerability of static logic controls is demonstrated.**
2. **Native code reverse engineering: Using decompilation tools, the structure of the C/C++ library is analysed to locate the hardcoded password.**
3. **Dynamic analysis: Via the Frida framework, memory manipulation of the application is achieved at runtime.**

The assignment aims to demonstrate that, in environments where code executes on the user's device, server-side validation and multi-factor authentication techniques are essential.

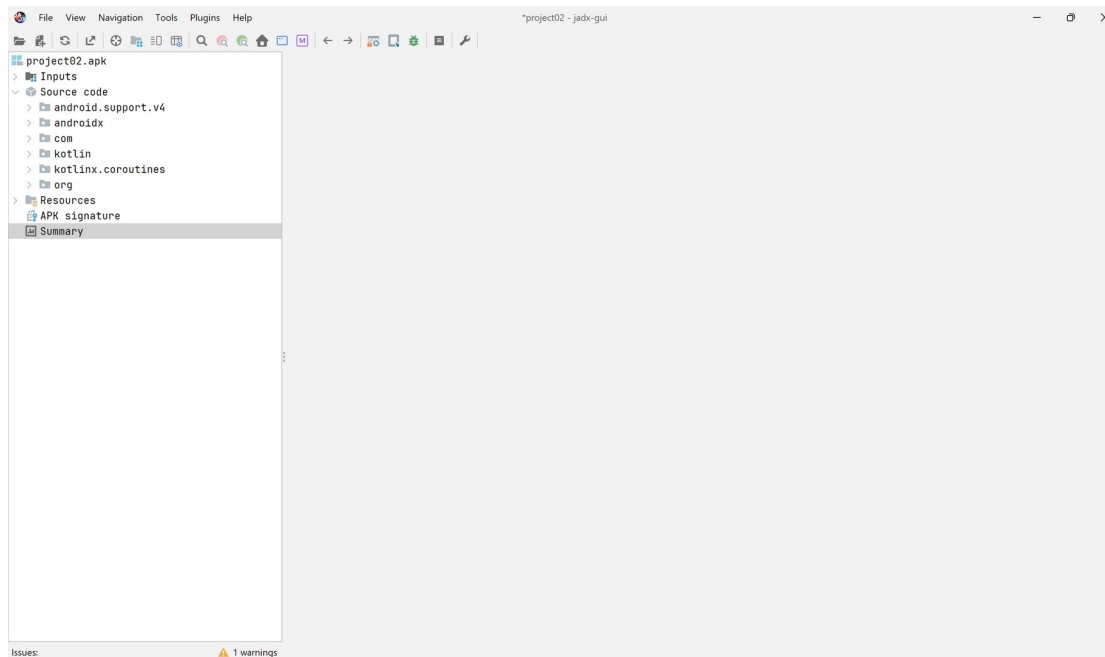
Practical part of the assignment

Method 1: JADX and APKTool (Smali Patching)

According to the assignment requirements (as defined in the README.txt file), the first approach requires bypassing the authentication check without knowing the correct credentials.

To achieve this, a static code analysis was first performed to fully understand the architecture and authentication logic of the application.

In contrast to the Apktool, which decompiles the application extracting code in Smali format (an intermediate, low-level representation of Dalvik bytecode), JADX performs reverse engineering at a higher level, producing readable Java code.



Next, the project02.apk installation file was imported into the tool's environment to begin the automatic reverse engineering process.

Via the navigation menu, the analysis focused on the source code directory and specifically on the package com.unipi.project02.

Examining the generated Java code of the MainActivity class, the login() method was identified — responsible for the authentication logic.

- **Username check:** The application retrieves the text entered by the user in the uname field and compares it (via the equals command) with the hardcoded string 'admin'.
- **Password check:** In parallel, it retrieves the password from the pass field and compares it with the value returned by the native library call getPassword().
- If the conditions match (if block), the application allows entry, displaying the success message 'Login Successful!'.
 - If the comparison fails (else block), the message 'Wrong Credentials!' is displayed and entry is rejected.

Taking into account that the goal of Method 1 is to bypass the check without using the correct password (which is still unknown at this stage), the following strategy was adopted: inverting the logical conditions within the Smali code.

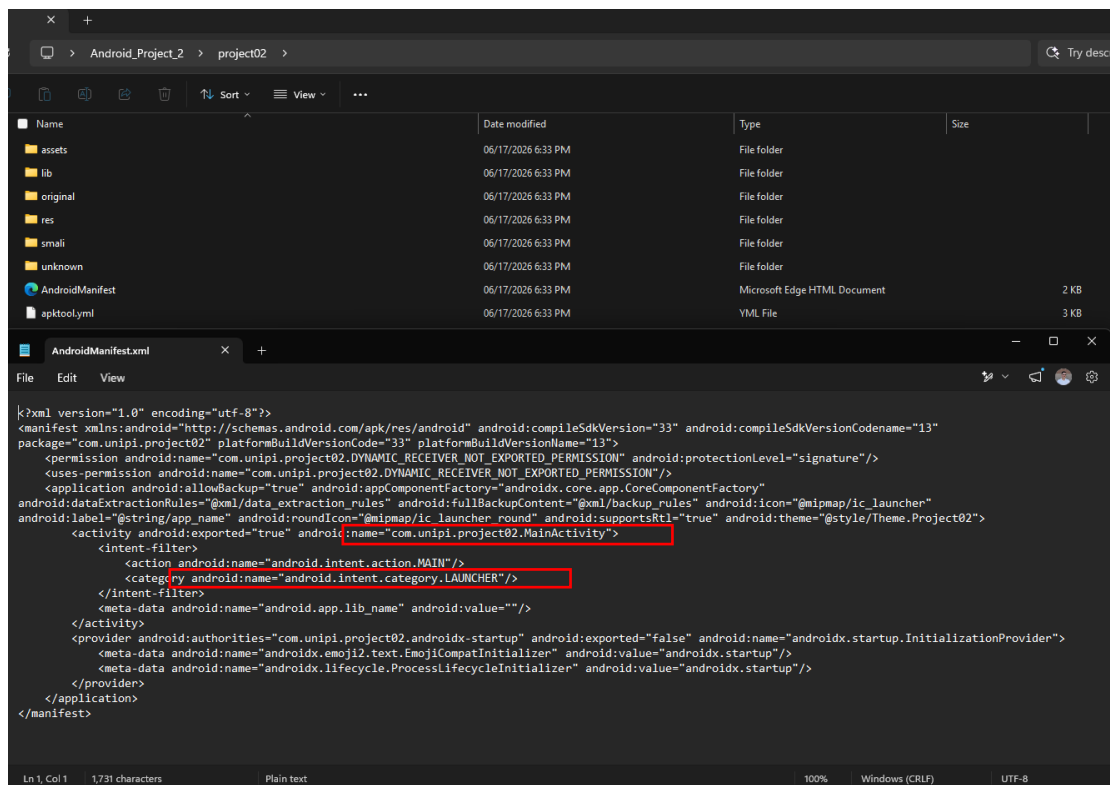
To perform the code change (patching), it was necessary to extract the compressed application files using Apktool.

The process was executed via the command-line interface (Command Prompt), using the command `apktool d project02.apk`.

```
C:\Users\Kalai\Desktop\Android_Project_2>apktool d project02.apk
I: Using Apktool 3.0.1 on project02.apk with 8 threads
I: Loading resource table...
I: Baksmaling classes.dex...
I: Decoding value resources...
I: Loading resource table from file: C:\Users\Kalai\AppData\Local\apktool\framework\1.apk
I: Decoding file resources...
I: Generating values XMLs...
I: Decoding AndroidManifest.xml with resources...
I: Copying original files...
I: Copying assets...
I: Copying lib...
I: Copying unknown files...

C:\Users\Kalai\Desktop\Android_Project_2>
```

On completion of the command execution, a new local directory (project02) was created containing the source code in Smali format.



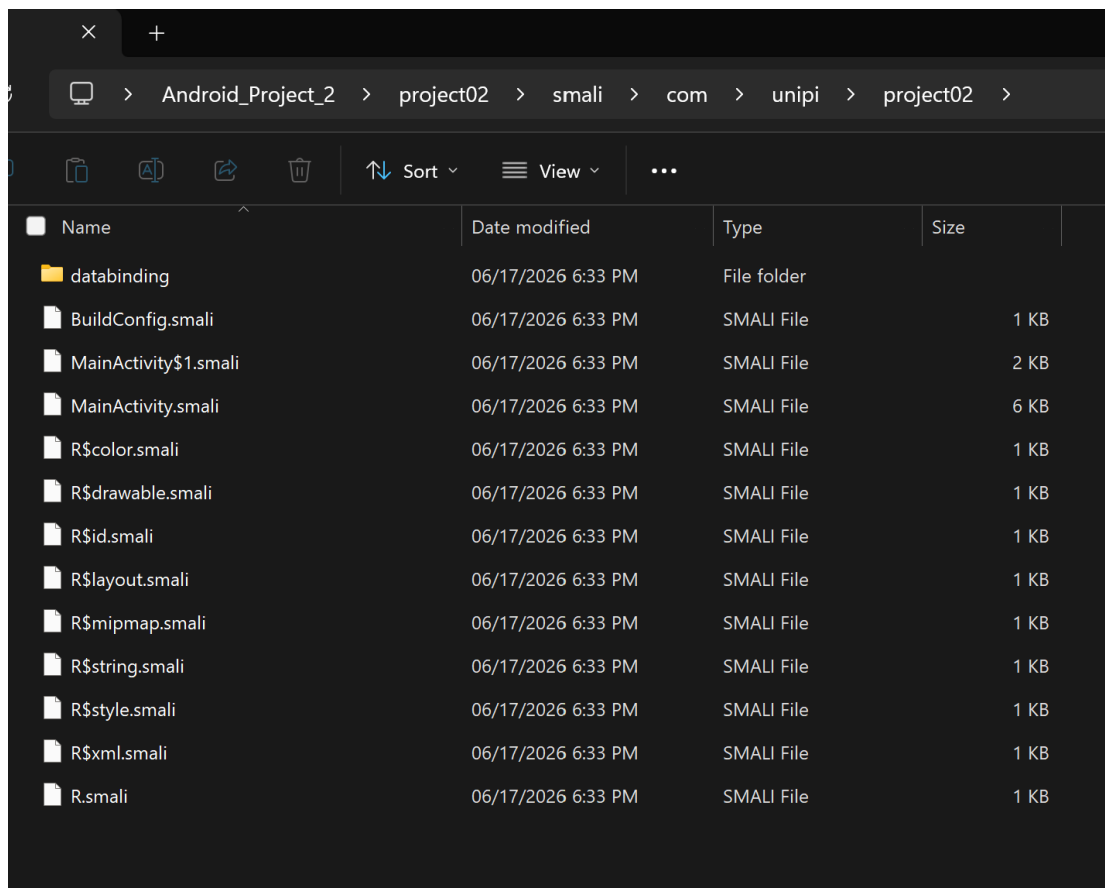
```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android" android:compileSdkVersion="33" android:compileSdkVersionCodename="13"
package="com.unipi.project02" platformBuildVersionCode="33" platformBuildVersionName="13">
    <permission android:name="com.unipi.project02.DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION" android:protectionLevel="signature"/>
    <uses-permission android:name="com.unipi.project02.DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION"/>
    <application android:allowBackup="true" android:appComponentFactory="androidx.core.app.CoreComponentFactory"
android:dataExtractionRules="@xml/data_extraction_rules" android:fullBackupContent="@xml/backup_rules" android:icon="@mipmap/ic_launcher"
android:label="@string/app_name" android:roundIcon="@mipmap/ic_launcher_round" android:supportRtl="true" android:theme="@style/Theme.Project02">
        <activity android:exported="true" android:name="com.unipi.project02.MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN"/>
                <category android:name="android.intent.category.LAUNCHER"/>
            </intent-filter>
            <meta-data android:name="android.app.lib_name" android:value=""/>
        </activity>
        <provider android:authorities="com.unipi.project02.androidx.startup" android:exported="false" android:name="androidx.startup.InitializationProvider">
            <meta-data android:name="androidx.emoji2.text.EmojiCompatInitializer" android:value="androidx.startup"/>
            <meta-data android:name="androidx.lifecycle.ProcessLifecycleInitializer" android:value="androidx.startup"/>
        </provider>
    </application>
</manifest>
```

From inspection of the XML code, two critical pieces of information were extracted:

1. The intent-filter containing the category '<category android:name="android.intent.category.LAUNCHER">' — identifying the application's entry point.
2. The declaration 'android:name="com.unipi.project02.MainActivity"' revealed the exact path and filename containing the main application logic.

Using this information, navigation was directed specifically to the path `project02/smali/com/unipi/project02/` to locate the `MainActivity.smali` file.

Inside the source code directory `project02/smali/com/unipi/project02/`, multiple files related to the application's classes were found.



Inside `MainActivity.smali`, the execution flow of `login()` is determined by two consecutive logical checks.

1. **Username check:** The application calls the `equals` method (which in Dalvik code translates as `invoke-virtual`), comparing the entered username with the hardcoded string `'admin'`.

2. **Password check:** If the first check completes successfully, the program proceeds to check the password. Here, a call is made to the native function `getPassword()` (invoke-static).

```
# virtual methods
.method public login()V
    .locals 3

    .line 47
    iget-object v0, p0, Lcom/unipi/project02/MainActivity;.>uname:Landroid/widget/EditText;
    invoke-virtual {v0}, Landroid/widget/EditText;.>getText()Landroid/text/Editable;
    move-result-object v0

    invoke-virtual {v0}, Ljava/lang/Object;.>toString()Ljava/lang/String;
    move-result-object v0

    const-string v1, "user457"

    invoke-virtual {v0, v1}, Ljava/lang/String;.>equals(Ljava/lang/Object;)Z
    move-result v0

    const/4 v1, 0x0
    if-eqz v0, :cond_0

    invoke-virtual {p0}, Lcom/unipi/project02/MainActivity;.>stringFromJNI()Ljava/lang/String;
    move-result-object v0

    iget-object v2, p0, Lcom/unipi/project02/MainActivity;.>pass:Landroid/widget/EditText;
    invoke-virtual {v2}, Landroid/widget/EditText;.>getText()Landroid/text/Editable;
    move-result-object v2

    invoke-virtual {v2}, Ljava/lang/Object;.>toString()Ljava/lang/String;
    move-result-object v2

    invoke-virtual {v0, v2}, Ljava/lang/String;.>equals(Ljava/lang/Object;)Z
    move-result v0

    if-eqz v0, :cond_0

    .line 48
    invoke-virtual {p0}, Lcom/unipi/project02/MainActivity;.>getApplicationContext()Landroid/content/Context;
    move-result-object v0

    const-string v2, "Login Successful!"
```

To completely bypass this mechanism, the Smali Patching technique was applied — replacing the conditional jump instructions (`if-eqz`) with their inverse (`if-nez`), effectively inverting the authentication logic.

Completing the Smali code modification, the next stage required rebuilding the decompressed directory back into a valid `.apk` file using the command `apktool b project02`.

```
C:\Users\Kalai\Desktop\Android_Project_2>apktool b project02 -o project02_patched.apk
I: Using Apktool 3.0.1 on project02.apk with 8 threads
I: Smaling smali folder into classes.dex...
I: AndroidManifest.xml and resources have not changed.
I: Building apk file...
I: Importing assets...
I: Importing lib...
I: Importing unknown files...
I: Built apk into: project02_patched.apk

C:\Users\Kalai\Desktop\Android_Project_2>
```

project02	06/17/2026 8:30 PM	File folder
Writeup_Template	06/12/2026 12:30 AM	File folder
apktool	06/17/2026 6:30 PM	Windows Batch File
apktool	06/17/2026 6:30 PM	Executable Jar File
project02.apk	06/12/2026 12:30 AM	APK File
project02_patched.apk	06/17/2026 8:33 PM	APK File
README	06/12/2026 12:30 AM	Text Document

Despite the successful rebuild, directly installing the project02_patched.apk file in the Android Studio environment failed — because the modified .apk file lacks a valid digital signature.

For this purpose, the standalone uber-apk-signer tool was used. By executing the command `java -jar uber-apk-signer.jar --apks project02_patched.apk`, the file was automatically signed with a development certificate.

```
C:\Users\Kalai\Desktop\Android_Project_2>java -jar uber-apk-signer-1.3.0.jar -a project02_patched.apk
source: C:\Users\Kalai\Desktop\Android_Project_2
binary-lib\windows-32_0_2\libwinpthread-1.dll
C:\Users\Kalai\AppData\Local\Temp\uapksigner-7386785299193876389
zipalign location: BUILT_IN
C:\Users\Kalai\AppData\Local\Temp\uapksigner-7386785299193876389\win-zipalign_33_0_2.exe15418115412522769127.tmp
keystore:
[0] e0a093b2 C:\Users\Kalai\.android\debug.keystore (DEBUG_ANDROID_FOLDERS)
01. project02_patched.apk

SIGN
file: C:\Users\Kalai\Desktop\Android_Project_2\project02_patched.apk (4.78 MiB)
checksum: 13198b625cbfa0dd1246a80d951d7e5e2f38dab366d4658e9925713a0073e0eb (sha256)
- zipalign success
WARNING: A restricted method in java.lang.System has been called
WARNING: java.lang.System::loadLibrary has been called by org.conscrypt.NativeLibraryUtil in an unnamed module (file:/C:/Users/Kalai/Desktop/Android_Project_2/uber-apk-signer-1.3.0.jar)
WARNING: Use --enable-native-access=ALL-UNNAMED to avoid a warning for callers in this module
WARNING: Restricted methods will be blocked in a future release unless native access is enabled

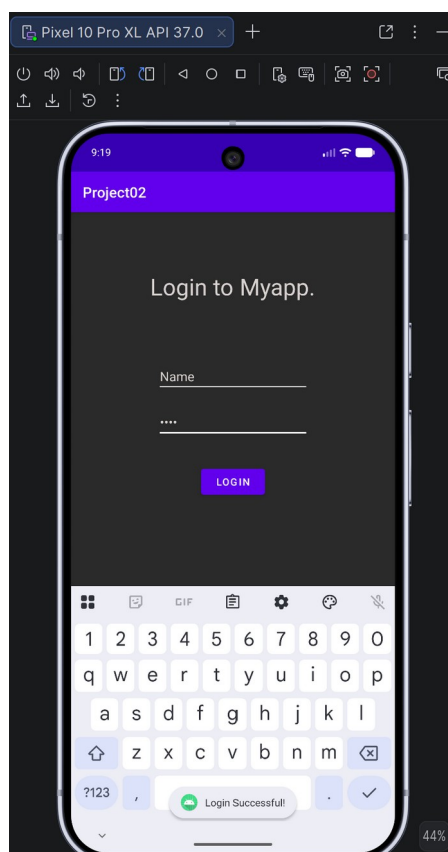
- sign success

VERIFY
file: C:\Users\Kalai\Desktop\Android_Project_2\project02_patched-aligned-debugSigned.apk (4.86 MiB)
checksum: d69d9810fd28e61cc2f3f1f474a5200a803fe76f226c3a7c0c6857d37736168b (sha256)
- zipalign verified
- signature verified [v1, v2, v3]
45 warnings
Subject: C=US, O=Android, CN=Android Debug
SHA256: ef963756f24f12bd2002539f3312d6c56aacb567c0586f635ba5d2ccd08ff7ea / SHA256withRSA
Expires: Sat Apr 22 11:44:24 EEST 2056

[Wed Jun 17 20:50:13 EEST 2026][v1.3.0]
Successfully processed 1 APKs and 0 errors in 1.11 seconds.
C:\Users\Kalai\Desktop\Android_Project_2>
```

The final signed file project02_patched-aligned-debugSigned.apk was installed on the virtual device (Android Emulator) via the Android Studio Device Manager.

Due to the prior checks (if-eqz to if-nez) the meaning of the reversed.

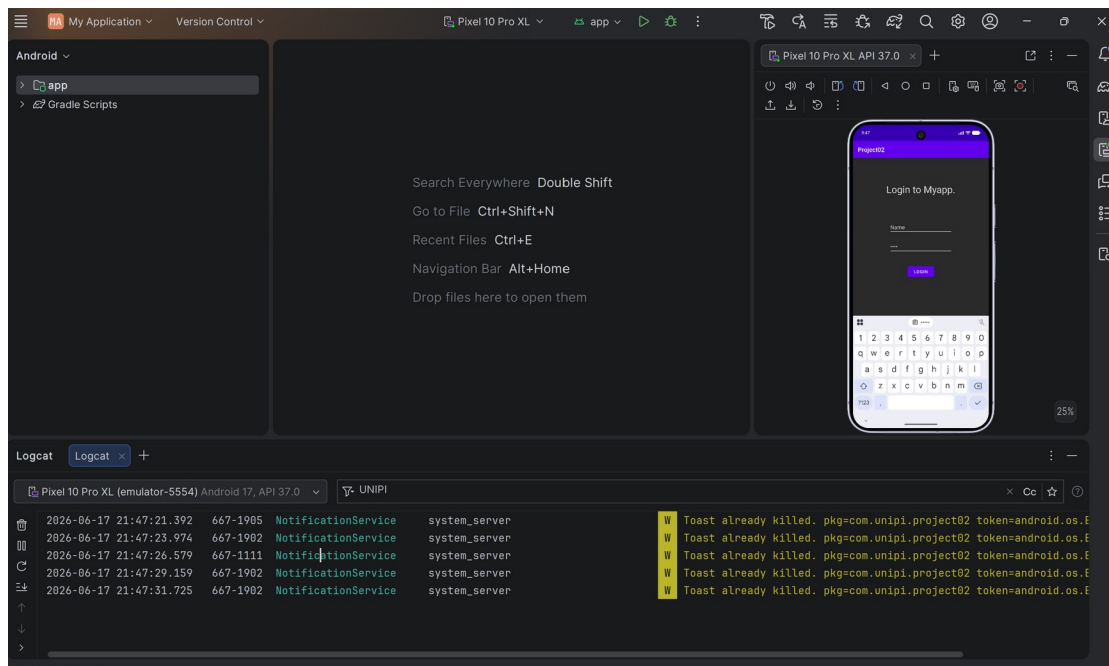


inversion of the logical inside the Smali code, comparisons was



During the successful execution of the bypass, it was observed that the application does not display the required flag (of the form UNIP1{...}) in a visible UI element.

Applying a search filter with the keyword 'UNIP1', the only findings concerned system notifications.

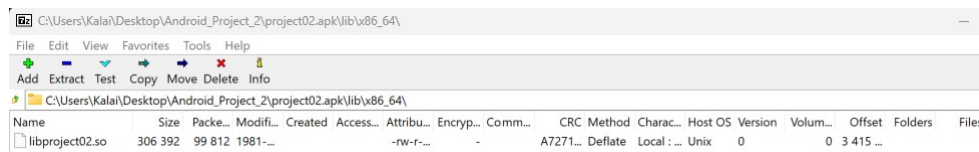


As confirmed from the Java source code of the application (in JADX), the mechanism simply prints the success message without displaying any flag.

Method 2: JADX and Ghidra (Credential extraction via reverse engineering)

To start the second method, which targets the actual recovery of the hidden password, the focus shifted to the native library of the application.

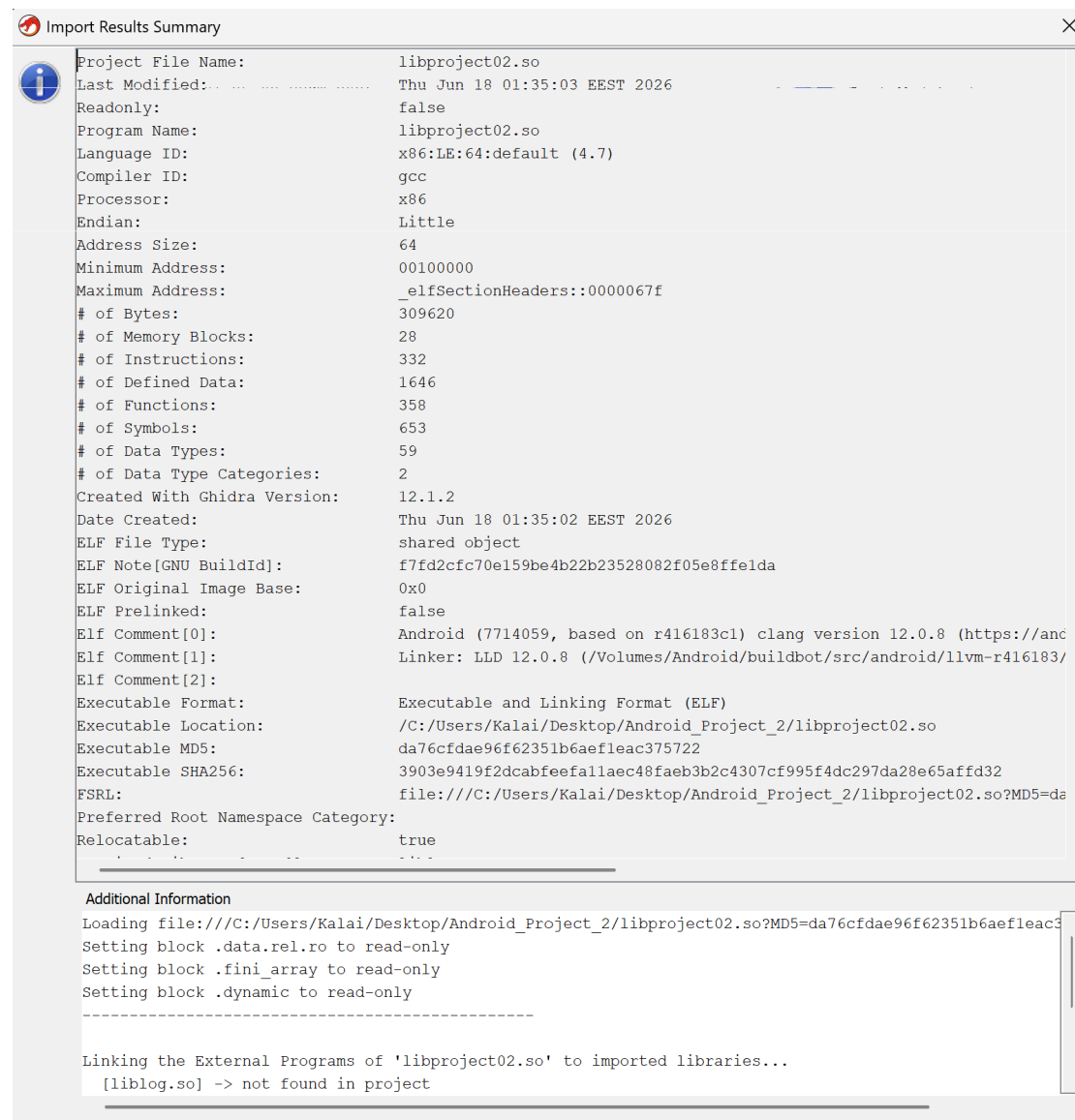
During the initial static analysis with the JADX tool, inside the MainActivity class the declaration 'System.loadLibrary("project02")' was identified — a clear indicator that part of the application's logic has been moved to a compiled C/C++ library (.so).



Name	Size	Packe...	Modifi...	Created	Access...	Attribu...	Encryp...	Comm...	CRC	Method	Charac...	Host OS	Version	Volum...	Offset	Folders	Files
libproject02.so	306 392	99 812	1981...			-rw-r--...	-		A7271...	Deflate	Local :...	Unix	0		0 3 415 ...		

This file was extracted to the local working directory, becoming the main target of the analysis. For the analysis of the binary file, the Ghidra reverse engineering platform (NSA) was used.

During import, the system successfully recognised the file structure as 64-bit ELF (Executable and Linkable Format), confirming it is a shared library compiled for the arm64-v8a architecture.



Import Results Summary

Project File Name: libproject02.so

Last Modified: Thu Jun 18 01:35:03 EEST 2026

Readonly: false

Program Name: libproject02.so

Language ID: x86:LE:64:default (4.7)

Compiler ID: gcc

Processor: x86

Endian: Little

Address Size: 64

Minimum Address: 00100000

Maximum Address: _elfSectionHeaders::0000067f

of Bytes: 309620

of Memory Blocks: 28

of Instructions: 332

of Defined Data: 1646

of Functions: 358

of Symbols: 653

of Data Types: 59

of Data Type Categories: 2

Created With Ghidra Version: 12.1.2

Date Created: Thu Jun 18 01:35:02 EEST 2026

ELF File Type: shared object

ELF Note[GNU BuildId]: f7fd2cfc70e159be4b22b23528082f05e8ffe1da

ELF Original Image Base: 0x0

ELF Prelinked: false

Elf Comment[0]: Android (7714059, based on r416183c1) clang version 12.0.8 (https://and

Elf Comment[1]: Linker: LLD 12.0.8 (/Volumes/Android/buildbot/src/android/llvm-r416183/

Elf Comment[2]:

Executable Format: Executable and Linking Format (ELF)

Executable Location: /C:/Users/Kalai/Desktop/Android_Project_2/libproject02.so

Executable MD5: da76cfdae96f62351b6aefleac375722

Executable SHA256: 3903e9419f2dcabfeefallaec48faeb3b2c4307cf995f4dc297da28e65affd32

FSRL: file:///C:/Users/Kalai/Desktop/Android_Project_2/libproject02.so?MD5=da

Preferred Root Namespace Category:

Relocatable: true

Additional Information

Loading file:///C:/Users/Kalai/Desktop/Android_Project_2/libproject02.so?MD5=da76cfdae96f62351b6aefleac3

Setting block .data.rel.ro to read-only

Setting block .fini_array to read-only

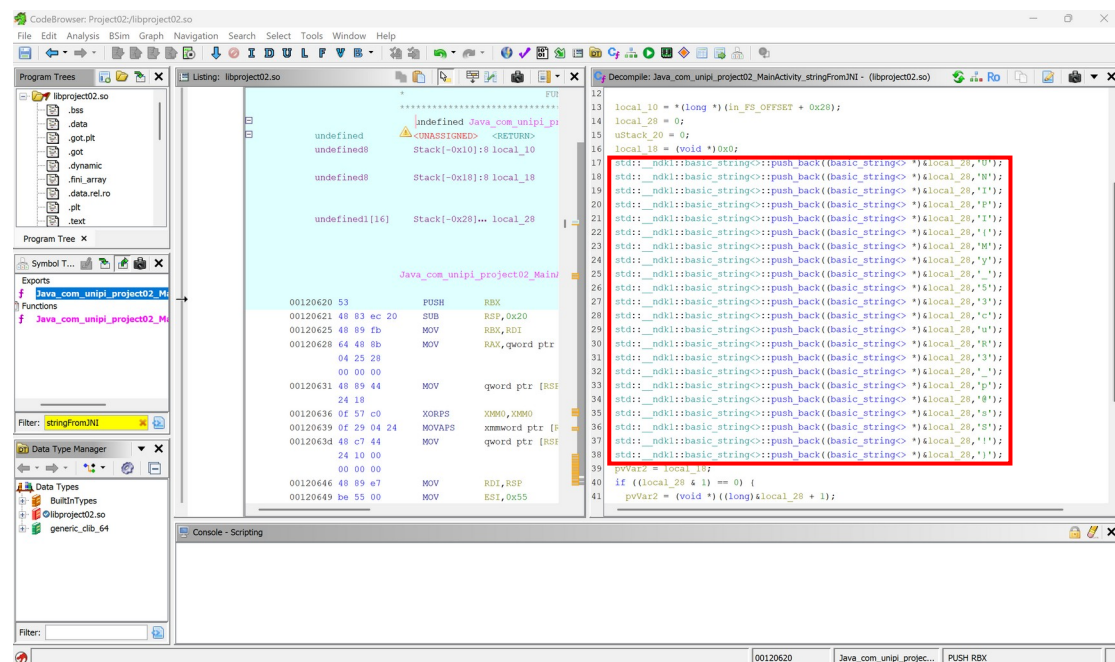
Setting block .dynamic to read-only

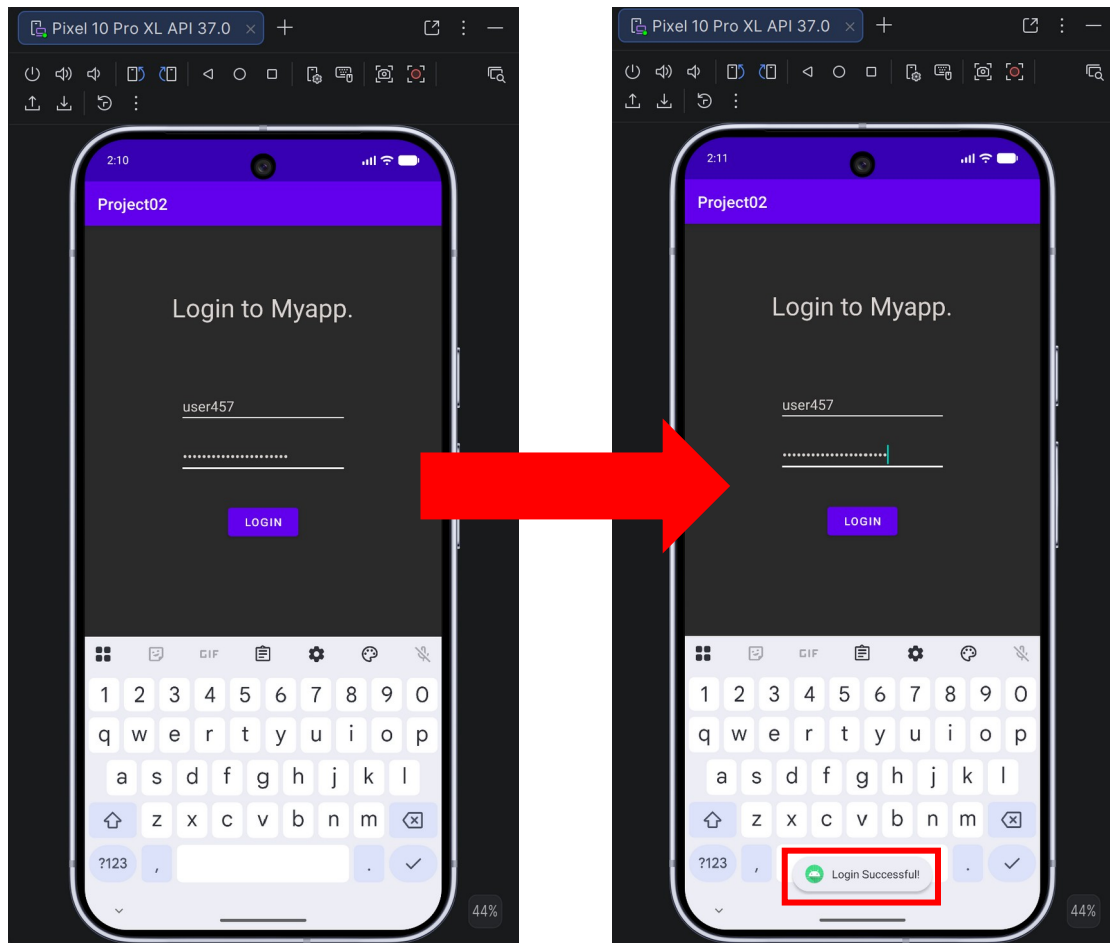
Linking the External Programs of 'libproject02.so' to imported libraries...

[liblog.so] -> not found in project

With the libproject02.so file open in Ghidra's main workspace, the automatic analysis process was executed.

From examining the source code, it was found that the application's author applied obfuscation techniques for password protection, using stack variables to store individual characters.

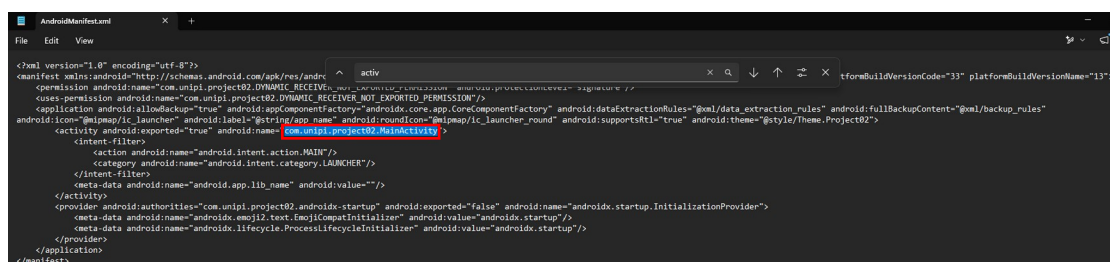




According to the assignment requirements in the README.txt file, three different approaches are requested for the login bypass.

Manifest Bypass Hypothesis

A common static bypass technique involves modifying the AndroidManifest.xml file (via Apktool). The basic idea is to redirect execution to an alternative activity that does not require authentication.



The absence of alternative activities technically rules out the Manifest Bypass possibility. The application does not route traffic to any secondary activity beyond MainActivity.

Method 3: Dynamic analysis with Frida.

Having examined the static intervention approaches (Smali Patching and Ghidra Decompilation), the third method focuses on dynamic analysis — interacting with the application while it runs in memory.

A basic prerequisite for starting the dynamic analysis is the installation of the authentic (unmodified) version of the application on a rooted virtual device.

During the initial privilege escalation attempt via the Android Debug Bridge (ADB) command-line tool, using the adb root command, a 'adb cannot run as root in production builds' error was returned.

The 'key' to success at this step was the strategic selection of the appropriate software version. Instead of production-ready images (Google Play variants), a development/testing image (Google APIs) was used.

```
C:\Users\Kalai\Desktop\Android_Project_2>C:\Users\Kalai\AppData\Local\Android\Sdk\platform-tools\adb root
restarting adbd as root
```

For the Frida framework to gain access to the operating system's processes, the presence of the frida-server service running with root privileges inside the virtual device is required.

1. **File transfer:** The adb push command was used to securely transfer the file from the local system to the virtual device.
2. **Permission assignment:** Since the Android operating system is based on the Linux kernel, the transferred file required granting executable permissions via chmod.
3. **Starting the service in the background:** To keep the server active without blocking the terminal session, it was started in the background with the & operator.

With the server in full readiness, the next stage involved writing the code that would perform the runtime interception.

```
if (Java.available) {
```

```

Java.perform(function () {
    console.log("[+] Dynamic instrumentation environment initialized successfully.");

    // Retrieve the application's class from memory
    var targetActivity = Java.use("com.unipi.project02.MainActivity");

    // Override the login() method by redefining its implementation
    targetActivity.login.implementation = function () {
        console.log("[+] Intercepted call to: MainActivity.login()");
        console.log("[+] Diverting execution flow to bypass hardcoded verification.");

        // Dynamically load the necessary Android SDK classes
        var androidToast = Java.use("android.widget.Toast");
        var javaString = Java.use("java.lang.String");

        // Get the current application's Context
        var currentContext = this.getApplicationContext();

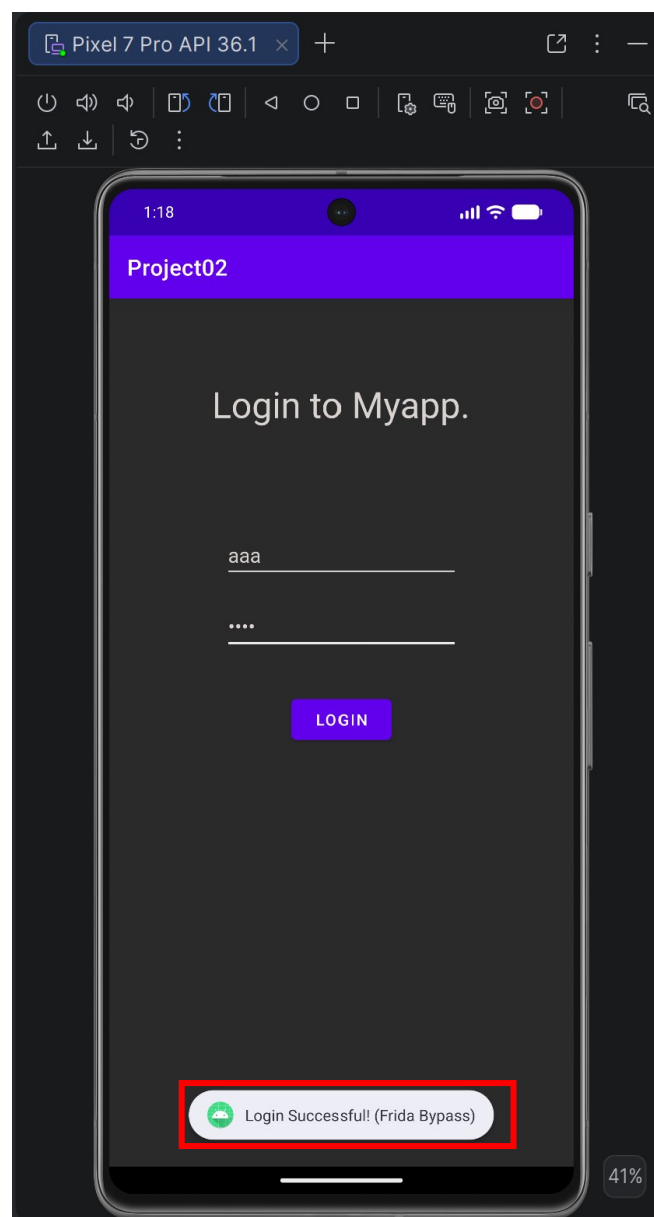
        // Construct and display the custom success message
        var bypassNotification = javaString.$new("Login Successful! (Frida Bypass)");
        androidToast.makeText(currentContext, bypassNotification, 0).show();

        console.log("[+] Custom notification injected. Authentication controls
bypassed.");
    };
});
}

```

The script first checks the availability of the Java environment (`Java.available`) and creates a safe execution thread (`Java.perform`). Inside the thread, it hooks the `MainActivity` class and overrides its `login()` method, replacing the original implementation.

With the bypass.js file saved in the working folder, a new terminal window was opened (keeping the frida-server running) and the script was injected into the running process. Once the application launched on the emulator's screen, a test was performed by entering random, invalid data in both the username and password fields.



This specific behaviour constitutes the undeniable confirmation of Method 3's success, demonstrating in practice that it is possible to completely bypass any authentication mechanism from outside the application.

Conclusion

This exercise highlighted the complexity but also the multiple vulnerabilities of authentication mechanisms in the Android ecosystem.

1. **Logic bypass (Smali Patching): Confirmed that relying on the static code flow is insecure, as the application's authentication logic can be inverted without knowing the credentials.**
2. **Native code reverse engineering (Ghidra): Proved that machine code, even with obfuscation techniques such as stack-based string assembly, can be systematically decompiled to recover hardcoded secrets.**
3. **Dynamic analysis (Frida): Demonstrated the power of runtime interception, where the application's logic can be completely replaced on-the-fly without any modification to the application files.**

Overall, the assignment confirms the necessity of multiple security layers, such as server-side validation and hardware-backed key storage for sensitive secrets.

Bibliography

- Releases · skylot/jadx. (n.d.). Retrieved from <https://github.com/skylot/jadx/releases>
- Releases · patrickfav/uber-apk-signer. (n.d.). Retrieved from <https://github.com/patrickfav/uber-apk-signer/releases>
- Releases · NationalSecurityAgency/ghidra. (n.d.). Retrieved from <https://github.com/NationalSecurityAgency/ghidra/releases>
- Releases · frida/frida. (n.d.). Retrieved from <https://github.com/frida/frida/releases>
- OWASP MASTG - OWASP Mobile Application Security. (n.d.). Retrieved from <https://mas.owasp.org/MASTG/>
- ApkTool | ApkTool. (n.d.). Retrieved from <https://ibotpeaches.github.io/Apktool/>
- Frida • A world-class dynamic instrumentation toolkit. (n.d.). Retrieved from <https://frida.re/>